The Model

A personalized, context-aware itinerary recommender — explained

The next-POI engine (GCN + GRU + user embedding + context), decoded into tourist itineraries by the Frozen Rollout

Companion document to the model-centric jury deck

Repository: github.com/6ym6n/PFE_IMPLEMTATION

Executive summary — the model in a nutshell

This document explains the deck I present to the jury. The headline is a single **model** and a single **method**. The model is a **next-POI engine**: a graph neural network (GCN) over a POI graph, a sequence model (GRU) over the visited route, a **user embedding** (the title's *user preferences*), and per-step **context** (Δdistance, Δtime — the title's *contextual data*), combined into a score over all POIs. The method is the **Frozen Rollout**: roll that engine out — repeatedly append the best next POI, masking visited ones and reserving the end — to build a full itinerary, with no extra training.

Everything else is a supporting experiment: training a dedicated *integrated* itinerary model (the Trained Pointer) — which did **not** beat the simpler Frozen Rollout; a literature-comparable benchmark on the Flickr datasets (the Flickr Benchmark) — which validates the approach and shows a simple Markov baseline is strong; and a personalization ablation. The takeaway I defend: **a strong, honest, personalized engine, decoded simply into itineraries — the best and most interpretable of the methods I built.**

1. The goal

A tourist with limited time wants a **personalized day plan**: an ordered route of POIs (places worth visiting). The plan must reflect **who they are** (preferences) and the **context** (distances, time). My approach does both with one model — a personalized next-POI engine — decoded into an itinerary.

2. The model in one picture

My **model** is the next-POI engine; my **itinerary method** (the Frozen Rollout) decodes it. The engine scores "what to visit next"; the Frozen Rollout calls it repeatedly to build the whole route.

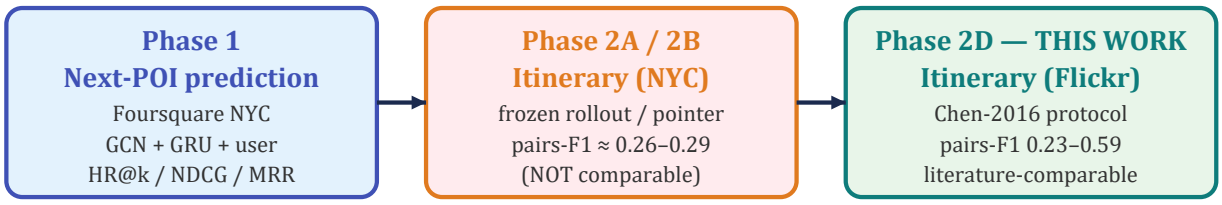

Figure 1 — The engine (the model) is reused everywhere: the Frozen Rollout decodes it into NYC itineraries; the Flickr Benchmark validates the family.

3. The model — architecture

The engine fuses three signals into a score over every POI. (1) A **POI graph** — places linked if geographically close or often visited together — encoded by a 2-layer **GCN** into a feature vector per POI. (2) Per-step **context** (Δd, Δt) encoded by small MLPs. (3) A **user embedding**. A **GRU** reads the route so far; an **MLP head** combines its state with the user to score the next POI.

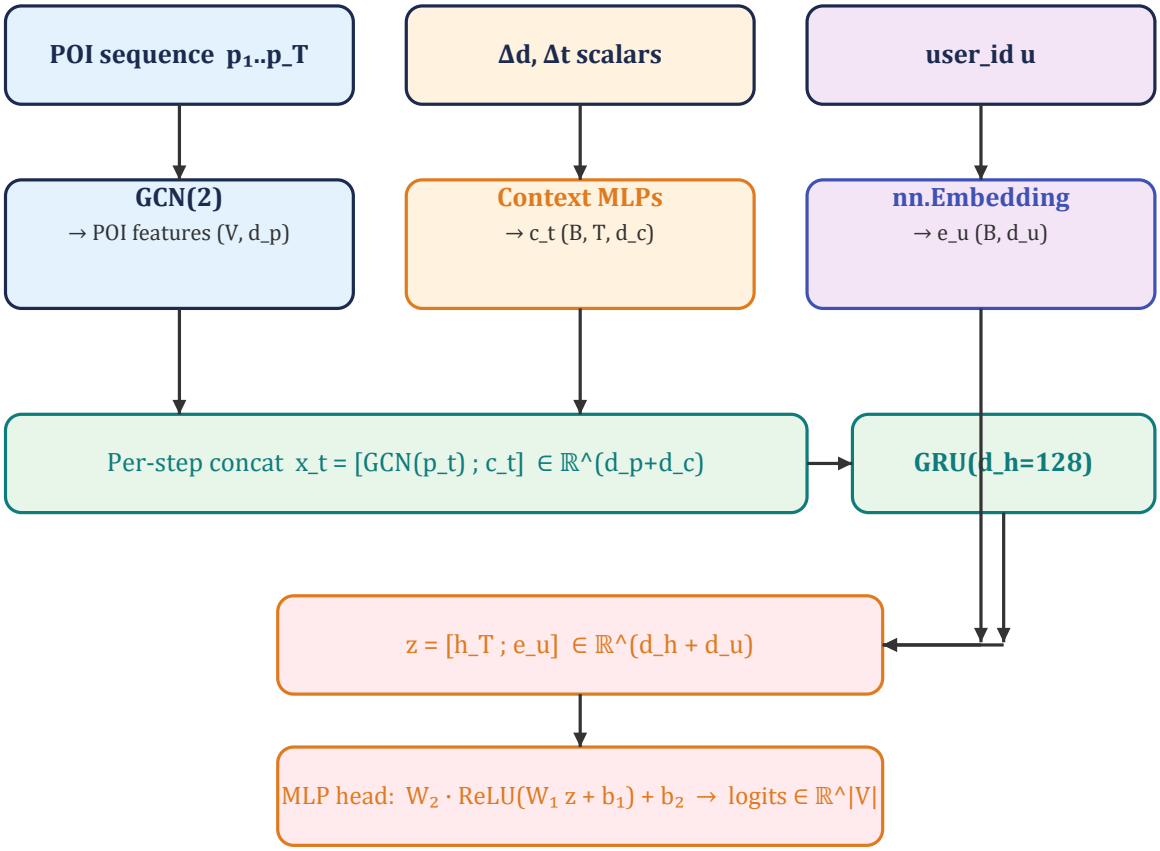

Figure 2 — The model I built: graph + context + user → GRU → a next-POI score. Trained once on Foursquare NYC; reused for every itinerary.

4. How each part works

- **POI graph + GCN.** Places are nodes; edges join nearby or co-visited POIs. The GCN mixes each POI's vector with its neighbours' (2 hops), giving each POI a "fingerprint" aware of its surroundings.
- **Context (Δd, Δt).** How far and how long since the previous place; small networks lift these scalars into vectors the model can use.
- **User embedding.** A small learned "taste profile" per tourist — the title's *user preferences*.
- **GRU sequence model.** Reads the places visited so far and summarises them into a single state vector.
- **Scoring head.** Combines the state and the user, and outputs a score for every POI — "what to visit next".

5. The model carries the thesis title

The title — "…Based on **User Preferences** and **Contextual Data**" — is a description of the architecture, not an aspiration: **user preferences** = the user embedding; **contextual data** = the Δd / Δt context features. Both live inside the model.

6. The engine works (Phase 1)

Before building itineraries, the engine must be a sound next-POI model. On Foursquare NYC (full vocabulary) it reaches **HR@1 = 0.187** (HR@10 = 0.588, MRR = 0.316) — the honest LSTM/STGCN tier of the LLM4POI benchmark. A strong, leakage-free engine.

Model	HR@1
LSTM	0.130
STGCN	0.180
Ours (GCN+GRU+user)	0.187
STAN	0.220
GETNext	0.240
STHGCN	0.270
LLM4POI	0.340

Table 1 — HR@1 tier (LLM4POI). Ours sits between STGCN and STAN.

7. Frozen Rollout — the itinerary method

The Frozen Rollout turns the engine into an itinerary by **rolling it out**: start at the start POI, ask the engine to score the next POI, block already-visited POIs (loop-free) and keep the end POI in reserve, append the best one, and repeat until the route has the desired length K and ends at the end POI. **No new training** — it is an inference-time decoder over the frozen engine, so the itinerary inherits the engine's personalization and context.

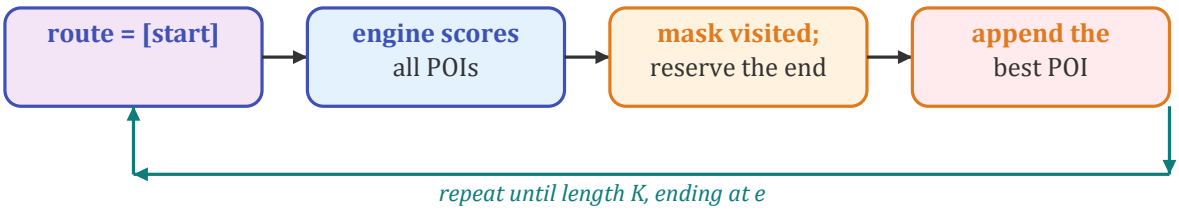

Figure 3 — Frozen Rollout: build the route one POI at a time by rolling out the engine. Always loop-free, exactly K stops, start→…→end.

8. Frozen Rollout — worked example (Edinburgh)

A real Edinburgh trip. The query gives start = 15, end = 16, K = 4; the engine fills the middle. Here it recovers the true route exactly:

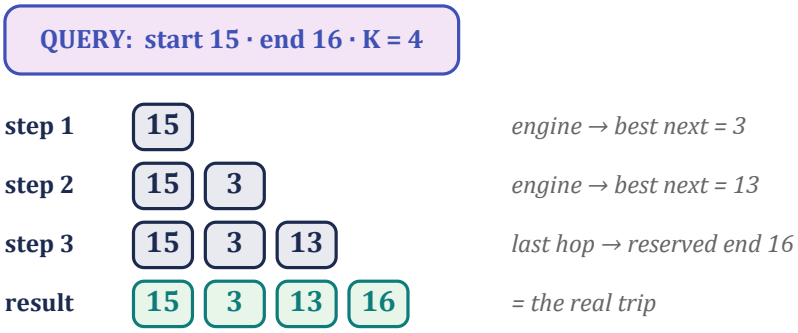

Figure 4 — The engine proposes POI 3, then 13; the last hop is the reserved end 16 → route [15, 3, 13, 16], matching the real trip (pairs-F1 = 1.0).

9. Frozen Rollout — results, and why it is the headline

On Foursquare NYC (length ≥ 3), the Frozen Rollout is the strongest of my itinerary methods — and crucially it **beats the dedicated integrated model** (the Trained Pointer):

Method (NYC, len≥3)	pairs-F1	kind
Frozen Rollout (greedy)	0.289	decoupled
Frozen Rollout (beam 3)	0.290	decoupled
Trained Pointer (no context)	0.259	integrated
Trained Pointer (+ context)	0.261	integrated

Decoding the strong engine beats training a separate model. The Frozen Rollout is the best (~0.29 pairs-F1), the simplest, and the most interpretable — and it reuses the personalized engine directly, so the itinerary inherits user preferences and context. That is why it is the model + method I lead with.

10. Validation — on the literature's scale

Because the NYC itinerary scores are not comparable to the published literature (a different benchmark), I re-run the family on the standard **Flickr** datasets under the exact published protocol (the Flickr Benchmark). A **Random baseline reproduces Chen 2016** (proving the setup is fair), and my methods land on the published 0.3–0.85 pairs-F1 scale — with the honest note that a simple **Markov** baseline is the strongest of mine on this tiny data.

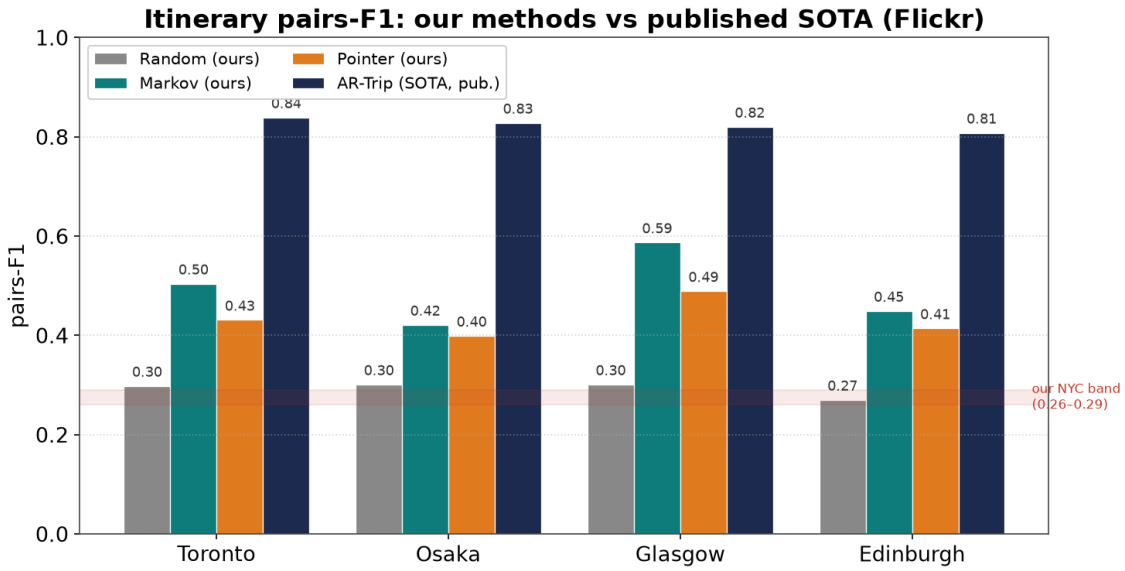

Figure 5 — Our methods beside the published SOTA. The Flickr Benchmark validates the approach; it does not replace the headline engine + Frozen Rollout.

11. What else I tested (rigor)

- **Integration (the Trained Pointer).** A dedicated pointer trained end-to-end did NOT beat the Frozen Rollout — the engine's far denser training signal wins. This directly tests Halder/DLIR's integration claim and nuances it.
- **Simple baselines.** On the small Flickr data, a Markov transition model is the strongest of my methods — quantified, not hidden.

Personalization. I switch the user embedding off vs on (identical otherwise) on the Flickr benchmark. The result is honest and mixed:

City	no user	with user	delta	recurring users
Osaka	0.442	0.435	-0.007	28%
Glasgow	0.451	0.489	+0.038	32%
Toronto	0.440	0.423	-0.017	55%
Edinburgh, Melbourne	—	—	(GPU run)	59%, 61%

A clear gain on Glasgow (+0.038), roughly neutral on Osaka and Toronto. The reason is **cold-start**: in leave-one-out even recurring users have very few trips, so the embedding is weakly estimated. *Personalization is a real lever, not a guaranteed win on small data* — and it is strongest where the engine (Foursquare, dense per-user history) lives.

12. Limitations & future work

- **Light context** — Δd/Δt only; add time-of-day, opening hours, queuing.
- **No scheduling** — add explicit time budgets (the DLIR direction).
- **Integrate properly** — share/warm-start the engine with the trained model to lift it past the Frozen Rollout.

• **Stronger personalization** — richer user features; finish the Edinburgh / Melbourne ablation on GPU.

13. In one sentence

A personalized, context-aware next-POI engine (GCN + GRU + user + context), decoded into tourist itineraries by rolling it out (the Frozen Rollout) — the strongest, simplest, and most honest of the methods I built; integration (the Trained Pointer) and the Flickr Benchmark are experiments that support it.